Birzeit University

Faculty of Engineering and Technology

Electrical and Computer Engineering Department

Data Base COMP333

Course Project - Sales System

Al Foosh Supermarket

Prepared By:
Noor Nasir 1220769
Masa Qadamani 1220196

Instructor: Dr. Naimeh al Hirbawi
Section:
Date: June. 10, 2025

Abstract

This project aims to design and develop a comprehensive database-driven website for a sales management system to streamline business operations, enhance decision-making, and improve efficiency. The system integrates key functionalities such as sales transaction processing, purchase tracking, employee administration, customer relationship management, and profit analysis. By maintaining a centralized database, the platform ensures real-time access to critical business data, including product details, customer purchase history, stock levels, and supplier information. Advanced features include automated sales reporting, profit margin calculations, employee performance tracking. The system provides an intuitive user interface for employees and managers to record transactions, manage customer interactions, and generate analytical insights, enabling data-driven business strategies. Built with security and scalability in mind, this sales management system serves as a powerful tool for businesses to enhance productivity, reduce operational costs, and improve overall profitability.

Table of Contents

Abstract 2

ER Diagram 4

First ERD 4

Updated ERD 5

SQL Relational Table Code 6

Normalization 11

Manager Requirements: 13

Teamwork: 13

Conclusion 14

Table Of Codes

Code 1 SQL Relational table 10

Table of Figures

Figure 1 First ERD 4

Figure 2 Updated ERD 5

ER Diagram

First ERD

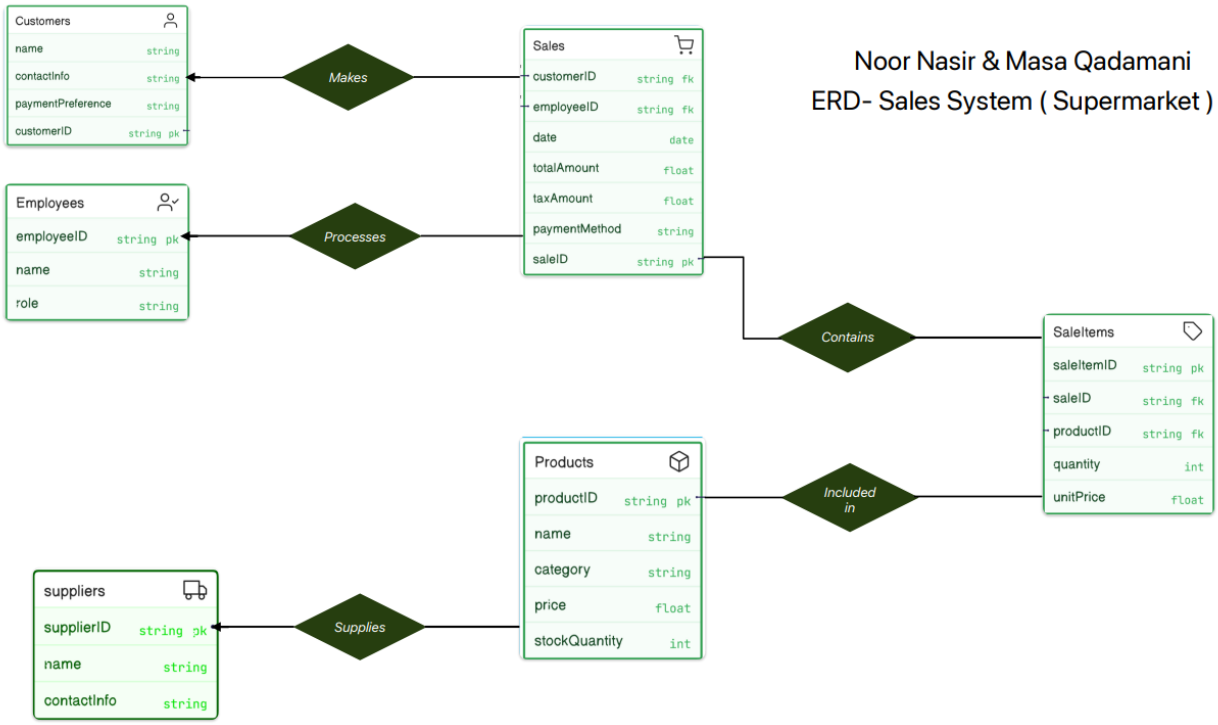


Figure 1 First ERD

As this is our first time implementing the system, we noticed that our initial diagram didn't fully cover all the manager's requirements. Therefore, we enhanced its functionality by adding new tables, refining relationships, and customizing queries to better meet the needs of the business.

as shown below:

Updated ERD

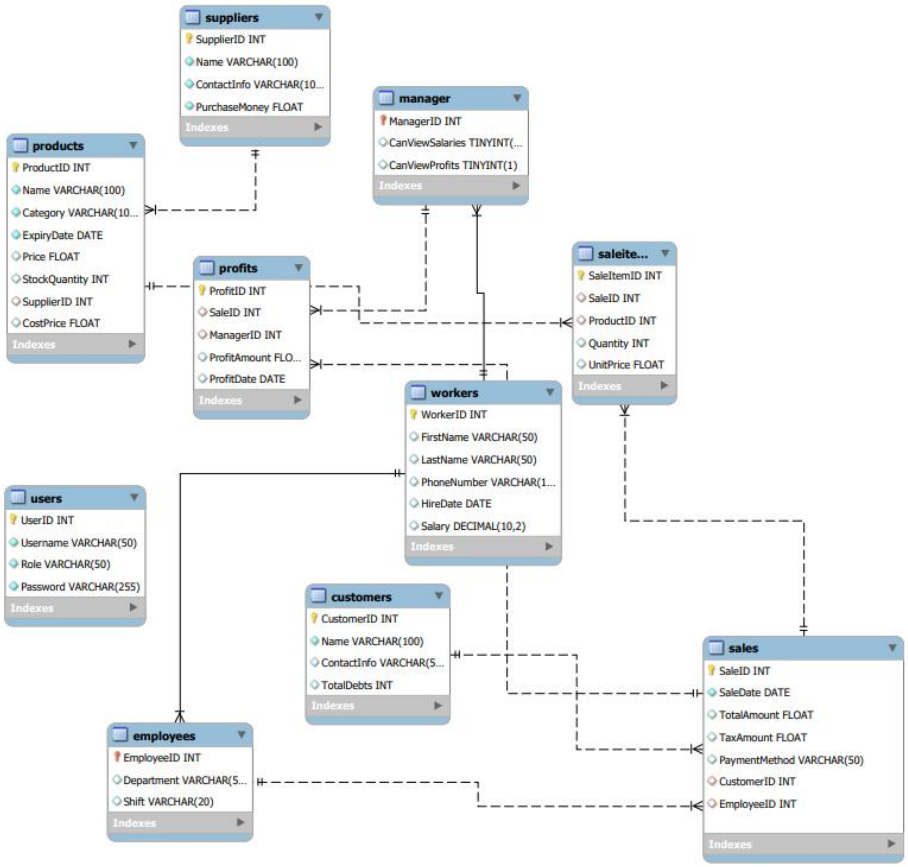


Figure 2 Updated ERD

The updated database schema represents a significant improvement over the initial design, offering better organization, clearer relationships, and more precise data handling.

We initially implemented two separate tables (Employees and Managers), we later realized it would be better to create two separate interfaces—one for managers and another for employees—so we updated the design so that we implemented an inheritance structure where both employees and managers inherited from a common worker class.

we also introduced a Profit table to track financial performance and align it with workforce management. This table links profits to specific sales and responsible managers while maintaining referential integrity.

And the Users table was added in order to identify the users of the system.

Overall, the updated diagram is more robust, user-friendly, and scalable, addressing the ambiguities and inefficiencies of the initial design while emphasizing clear relationships and practical business needs.

SQL Relational Table Code

```
-- Customers table
CREATE TABLE Customers (
    CustomerID INT AUTO_INCREMENT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    ContactInfo VARCHAR(50),
    TotalDebts INT DEFAULT 0
);
-- Workers table
CREATE TABLE Workers (
    WorkerID INT AUTO_INCREMENT PRIMARY KEY,
    FirstName VARCHAR(50),
    LastName VARCHAR(50),
    PhoneNumber VARCHAR(15),
    HireDate DATE,
    Salary DECIMAL(10, 2)
);
-- Employees table (extends Workers)
CREATE TABLE Employees (
```

```
    EmployeeID INT PRIMARY KEY,
    Department VARCHAR(50),
    Shift VARCHAR(20),
    FOREIGN KEY (EmployeeID) REFERENCES Workers(WorkerID) ON DELETE CASCADE
);
-- Manager table (extends Workers)
CREATE TABLE Manager (
    ManagerID INT PRIMARY KEY,
    CanViewSalaries BOOLEAN DEFAULT TRUE,
    CanViewProfits BOOLEAN DEFAULT TRUE,
    FOREIGN KEY (ManagerID) REFERENCES Workers(WorkerID) ON DELETE CASCADE
);
-- Suppliers table
CREATE TABLE Suppliers (
    SupplierID INT AUTO_INCREMENT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    ContactInfo VARCHAR(100) NOT NULL,
    PurchaseMoney FLOAT NOT NULL
);
-- Products table with foreign key to Suppliers
CREATE TABLE Products (
    ProductID INT AUTO_INCREMENT PRIMARY KEY,
```

```
    Name VARCHAR(100) NOT NULL,
    Category VARCHAR(100) NOT NULL,
    ExpiryDate DATE NOT NULL,
    Price FLOAT,
    StockQuantity INT,
    SupplierID INT,
    CostPrice FLOAT,
    FOREIGN KEY (SupplierID) REFERENCES Suppliers(SupplierID) ON DELETE CASCADE
);
-- Sales table with foreign keys to Customers and Employees
CREATE TABLE Sales (
    SaleID INT AUTO_INCREMENT PRIMARY KEY,
    SaleDate DATE NOT NULL,
    TotalAmount FLOAT,
    TaxAmount FLOAT,
    PaymentMethod VARCHAR(50),
    CustomerID INT,
    EmployeeID INT,
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID) ON DELETE CASCADE,
    FOREIGN KEY (EmployeeID) REFERENCES Employees(EmployeeID)
);
-- SaleItems table linking Sales and Products
```

```
CREATE TABLE SaleItems (
    SaleItemID INT AUTO_INCREMENT PRIMARY KEY,
    SaleID INT,
    ProductID INT NULL,
    Quantity INT,
    UnitPrice FLOAT,
    FOREIGN KEY (SaleID) REFERENCES Sales(SaleID) ON DELETE CASCADE,
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID) ON DELETE SET
NULL
);
-- Profits table linked to Sales and Manager
CREATE TABLE Profits (
    ProfitID INT AUTO_INCREMENT PRIMARY KEY,
    SaleID INT,
    ManagerID INT,
    ProfitAmount FLOAT,
    ProfitDate DATE,
    FOREIGN KEY (SaleID) REFERENCES Sales(SaleID) ON DELETE CASCADE,
    FOREIGN KEY (ManagerID) REFERENCES Manager(ManagerID)
);

-- Users table for login
CREATE TABLE Users (
    UserID INT AUTO_INCREMENT PRIMARY KEY,
```

```
    Username VARCHAR(50) UNIQUE NOT NULL,
    Role VARCHAR(50) NOT NULL,
    Password VARCHAR(255) NOT NULL
);
```

Code 1 SQL Relational table

This SQL script creates a full relational database the management system. Customers, members, managers, suppliers, products, sales, profits, and users are all defined, along with the proper relationships and limitations (such as foreign keys and cascade deletes). Following the creation of the schema, sample data is added to the tables for analysis and testing. Use cases such as sales reporting, profit analysis, and customer behavior are covered by the script's extensive collection of almost 25 queries, which range from simple data retrieval to more intricate aggregations and joins. It is intended to replicate a real-world context for retail operations management.

Normalization

In general, our database schema fits to the third normal form (3NF) and is well-normalized. effectively grouped data into atomic tables to prevent duplication, guaranteeing that every table has precisely specified facts pertaining to a specific goal. Simple tables with no recurring groupings, such as Customers, Suppliers, and Products, make explicit use of foreign keys when necessary. Effective generalization and specialization are used in the Workers, Employees, and Manager tables to prevent the duplication of similar information. Flexibility is increased by the Sales and SaleItems structure, which accurately models a one-to-many relationship. Last but not least, the Users and Profits tables respect to normalization rules while storing atomic and conceptually separate data. Overall, the design successfully uses primary and foreign keys, avoids redundant data storage, and preserves data integrity, exhibiting sound normalization principles.

Here's how our database schema meets each normalization form:

First Normal Form (1NF):

It is evident that every table stores atomic values in every column, which means that there is only one indivisible value at each row-column intersection. Fields don't contain any arrays or repetitive groupings. For instance, Customers store basic properties like Name and TotalDebts, and Products store simple variables like Price and ExpiryDate.

Second Normal Form (2NF):

The tables use single-column primary keys or proper composite keys (if applicable), ensuring all non-key attributes fully depend on the entire primary key. For example, in the SaleItems table, each item fully depends on its primary key, SaleItemID, and relationships with other tables are explicitly defined via foreign keys. There is no partial dependency, as each attribute clearly and fully depends on its primary key.

Third Normal Form (3NF):

By guaranteeing that non-primary properties are independent of one another, each table removes transitive dependencies. For example, instead of embedding supplier details within the Products table, the Products table links directly to suppliers using SupplierID. Similarly, the Sales and Employees tables avoid indirect dependencies by storing references (foreign keys) instead of duplicate non-primary data.

By employing atomic fields, avoiding partial dependencies, and getting rid of transitive dependencies with the right usage of foreign keys and well-structured tables, the schema generally complies with the 1NF, 2NF, and 3NF principles.

Manager Requirements:

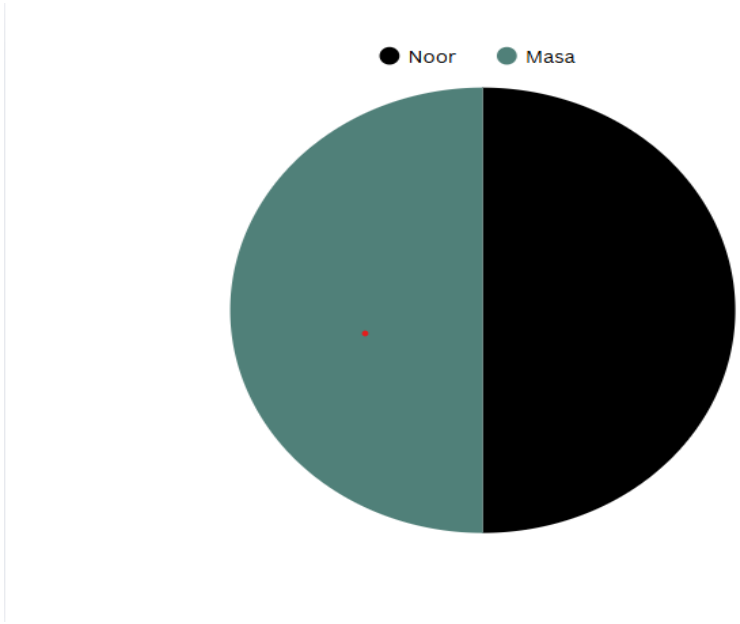
Although we implemented all the queries planned in Phase One, the manager's feedback is crucial. Based on their input, here are the additional requirements they requested:

1. **Ease of use:** The system should be simple so my employees can quickly learn and use it without much training.
2. **Real-time stock alerts:** to know when products run low, helping us avoid empty stocks.
3. **Easy-to-read financial reports:** Clear summaries showing profits, losses, and daily sales to quickly understand business health.
4. **Adaptability:** The system should easily grow and change as my supermarket expands.
5. **Compatibility:** It should smoothly connect with payment systems and tools already used in the store.
6. **Customer buying trends:** Insights into what customers prefer and buy most, helping me make smarter stocking and marketing decisions.

and all of them were implemented as required!!

Teamwork:

Success in teamwork comes when no one counts who did more.



Conclusion

This project showcases a successful implementation of a database-driven sales management system designed to meet the practical needs of Al Foosh Supermarket. From managing customers and employees to tracking sales, products, and profits, the system effectively brings together all key aspects of retail operations into one centralized platform. The improved ERD, normalized schema, and manager-driven enhancements demonstrate a deep understanding of both technical and business requirements. With features like role-based access, real-time reporting, and streamlined workflows, the system not only improves operational efficiency but also lays a strong foundation for future scalability. Overall, the solution is well-structured, user-friendly, and ready for real-world deployment.